A Brief Intro to NN Verification

Син Сяофан

Московский государственный университет имени М. В. Ломоносова
Факультет вычислительной математики и кибернетики

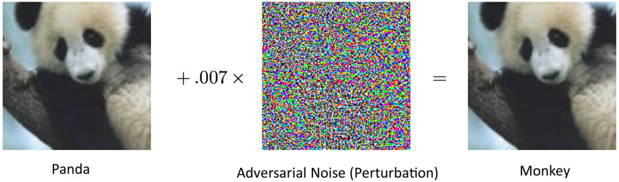
November 2025

Contents

Contents

Vulnerability of Neural Networks

- Neural networks have become a crucial element in modern artificial intelligence.
- However, they are often black-boxes and can behave unexpectedly and produce surprisingly wrong results, such as **adversarial examples**:

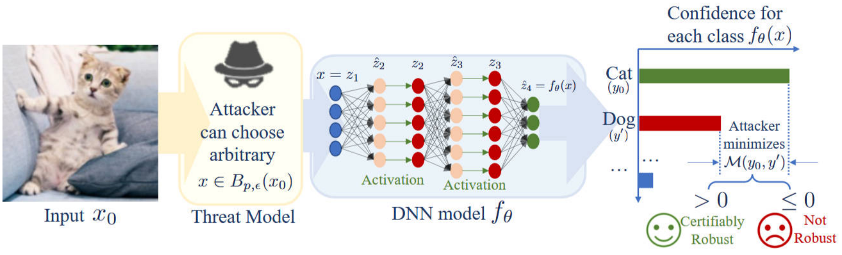


Vulnerability of Neural Networks

- In this case, we introduce the neural network verification problem, which aims to formally guarantee **properties** of **neural networks** such as **robustness**, safety, and correctness.
- **Robustness** means a neural network's ability to resist small, intentional manipulations of its input designed to trick this neural network.

Vulnerability of Neural Networks

- As the diagram shows, an attacker can create a slightly altered version of an input image (like the cat).
- A model is considered robust for that input if its confidence in the correct class ("Cat") remains higher than any incorrect class ("Dog").
- The verification process provides a guarantee that the neural network's prediction will not change within the specified threat boundaries, ensuring reliable and secure performance.

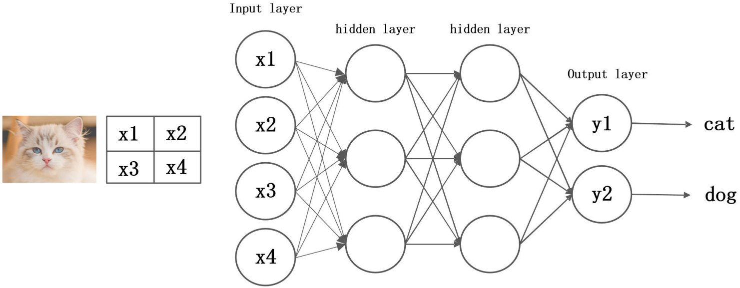


Contents

Neural Network Verification Workflow

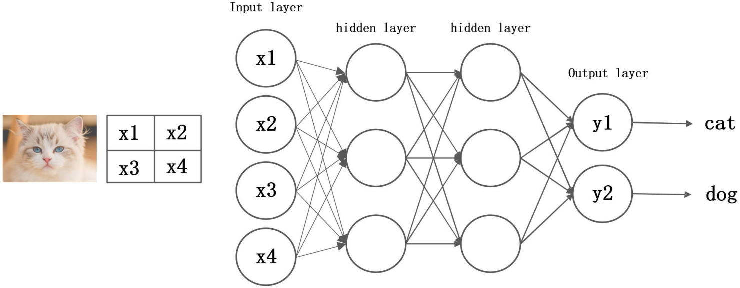
To help explain the Workflow of Neural Network Verification, let's consider a highly simplified scenario:

- Suppose the image of a cat is represented by only four pixel values—x_1, x_2, x_3, and x_4. These four pixel values serve as the input to the neural network.
- For the neural network to correctly classify the image as a cat, the output value y_1 (representing "cat") should be greater than y_2 (representing "dog").



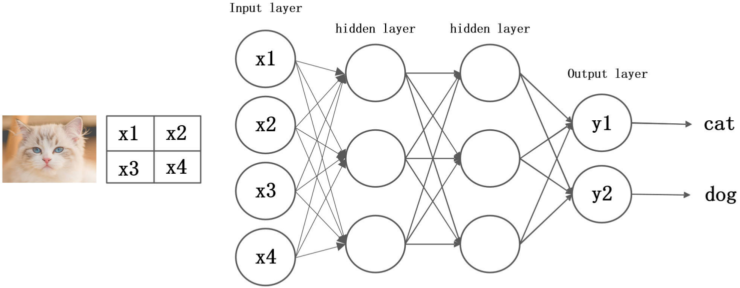
Neural Network Verification Workflow

- Assume that: $x_1 = 0.3, x_2 = 0.2, x_3 = 0.7, x_4 = 0.5$
- An attacker might subtly alter these pixel values in an attempt to fool the neural network into misclassifying the cat as a dog.
- Even with just four pixels, there are countless possible perturbations—some of which may result in the image being misclassified.



Neural Network Verification Workflow

- To assess the robustness of this neural network, we turn to neural network verification.
- For example, we may want to be absolutely certain whether modifying each pixel within a range of ± 0.02 could cause misclassification.
- Exhaustively testing all possible pixels within this range is computationally infeasible.

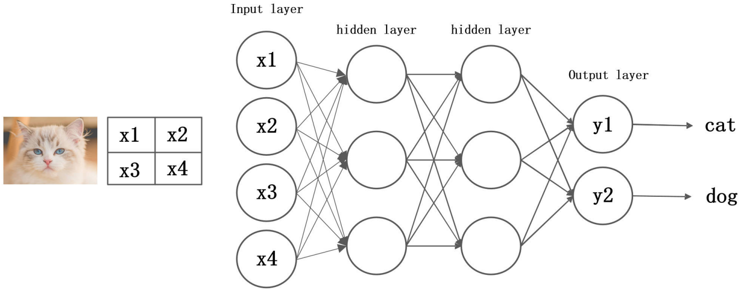


Neural Network Verification Workflow

- Instead, we set each pixel as an interval:

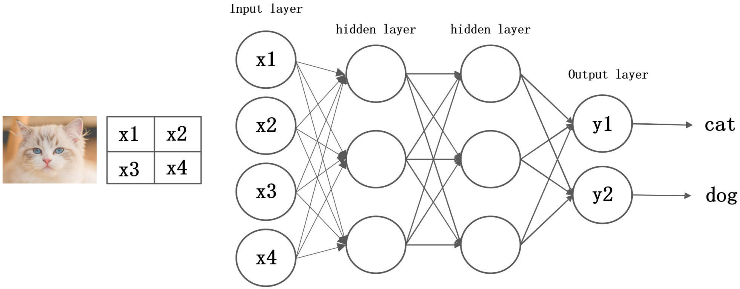
$$x_1 = [0.28, 0.32] \qquad x_2 = [0.18, 0.22] \qquad x_3 = [0.68, 0.72] \qquad x_4 = [0.48, 0.52]$$

- These intervals are then propagated through the neural network.
- Since all calculations are performed on intervals, the resulting outputs y_1 and y_2 will also be intervals.



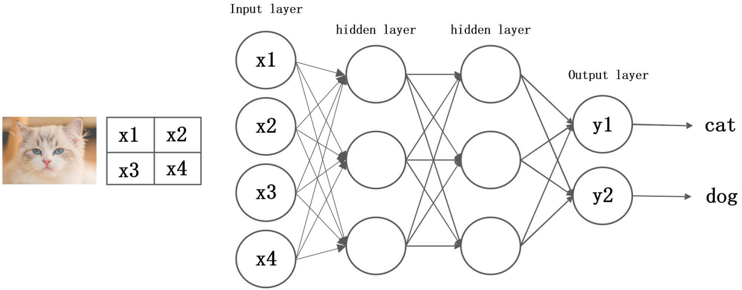
Neural Network Verification Workflow

- If the neural network is robust to perturbations within the ±0.02 range for this cat image, then the lower bound of the y_1 interval should still be greater than the upper bound of the y_2 interval.
- This guarantees that the neural network will consistently classify the image as a cat across all possible ±0.02 range perturbations.



Neural Network Verification Workflow

- The above illustrates the general workflow of neural network verification.
- A verifier is essentially a tool that can perform neural network verification workflow.
- It's worth to note that this neural network verification example is intentionally simplified for clarity and does not capture all technical nuances of formal verification methods.



Contents

Terminology and Evaluation Metrics Explanation

Terminology and Evaluation Metrics Explanation

- **Instance**—An instance consists of a neural network, an input image, a property specification (e.g. Robustness), and a timeout.
- For example, one instance might consist of a classifier for dog and cat, a cat image, a given perturbation range ±0.02, the requirement that output y_1 ("cat") should be greater than y_2 ("dog"), and a specific timeout.

- **Benchmark**—A benchmark consists of a set of related instances.
- Therefore, the benchmark provides predefined networks, images, property specifications, and timeouts. To evaluate a verifier, we simply run it on the benchmark without defining these elements ourselves.

Terminology and Evaluation Metrics Explanation

Terminology and Evaluation Metrics Explanation

- **Verified**—The verifier successfully proves that for all pixel perturbations within the ±0.02 range, the neural network consistently outputs "cat".
- This is the ideal outcome. It means the verifier can deterministically confirm that the neural network operates safely under the specified threat model (such as pixel perturbations within the ±0.02).
- Verifiers such as α-β-CROWN will give the number of successfully verified instances.

Terminology and Evaluation Metrics Explanation

Terminology and Evaluation Metrics Explanation

- **Falsified**—The verifier successfully finds a counterexample—such as a specific set of pixel values (e.g., x_1=0.31, x_2=0.19, . . .)—that causes the neural network to misclassify the "cat" as a "dog".
- This is also a valuable result. It shows that the verifier acts like a detective, uncovering concrete evidence that can "fool" the neural network. This demonstrates the tool's strong attack discovery capabilities.
- Verifiers such as α-β-CROWN will give the number of successfully falsified instances.

Terminology and Evaluation Metrics Explanation

Terminology and Evaluation Metrics Explanation

- **Incorrect Result**—The verifier returns an erroneous conclusion. For example, it may report the network as “safe” (Verified) when a counterexample actually exists (which should have been Falsified).
- This is a negative metric. It indicates potential flaws or inaccuracies in the verifier itself. The fewer incorrect results, the higher the reliability of the verifier.